OpenShift Networking – Learning Guide

Table of Contents

27. Securing Service-to-Service Communication
28. Monitoring and Alerts for Networking
29. Exposing Internal Services Securely
30. Pre-installation Networking Checklist
31. Overview of OpenShift v4 Networking
32. Importance of Networking in OpenShift
33. Key Networking Components
34. Network Plugins in OpenShift
35. Pod Network Definition
36. Definition of a Service
37. Types of Services
38. DNS Functionality
39. Need for Internal DNS
40. DNS Configuration Considerations
41. Role of Load Balancer
42. Ingress Controller Functionality
43. Load Balancer Options
44. Required Ports
45. Importance of Port Requirements
46. Infra Team Networking Considerations
47. Default MTU Size
48. NetworkPolicy Definition
49. Wildcard DNS Requirement
50. Multiple Ingress Controllers

---

1. What is the role of OVN-Kubernetes in OpenShift v4 networking?

OVN-Kubernetes is the default and recommended network plugin in OpenShift v4. It offers enhanced scalability, security, and features like egress IP, network policies, and IPv6 support. OVN leverages Open vSwitch and supports native Kubernetes NetworkPolicy objects. It enables more efficient flow management and separation of concerns. Infra teams benefit from simplified troubleshooting, better performance, and reduced network latency due to its distributed architecture. OVN also allows for hybrid networking scenarios, making it future-proof for cloud-native deployments.

2. Why is a unique pod CIDR important in OpenShift networking?

A unique pod CIDR ensures that each pod across the cluster gets a non-conflicting IP address. This is critical for routing traffic correctly between nodes and maintaining a flat networking model where every pod can talk to any other pod. Overlapping or misconfigured CIDRs lead to traffic drops, name resolution issues, and unpredictable behavior. From an infra perspective, careful planning of pod and service CIDRs is necessary to avoid overlap with the on-premises or cloud VPC IP ranges.

3. How does OpenShift handle ingress and egress traffic?

OpenShift uses Ingress Controllers (usually HAProxy-based) to manage ingress traffic, routing HTTP/HTTPS requests to internal services. For egress traffic (outbound from pods), OpenShift uses NAT through nodes or can be configured with egress firewalls or egress IPs using OVN. Proper handling of ingress/egress is crucial for application availability, security, and auditability. Infra teams often configure external firewalls and load balancers in coordination with OpenShift ingress and egress policies to maintain secure and compliant architectures.

4. What is the significance of the Cluster Network and Service Network in OpenShift?

The Cluster Network is used to allocate IPs for pods, while the Service Network is for ClusterIP services. These two non-overlapping CIDRs are defined during cluster installation and are immutable afterward. Their correct configuration ensures that pods can communicate within the cluster and services can route requests reliably. Poorly planned CIDRs may conflict with external IP ranges, requiring reinstallations. Hence, infra architects must size and scope these networks based on current needs and future scaling.

5. Why is DNS resolution crucial for service discovery in OpenShift?

OpenShift relies heavily on internal DNS to allow pods and services to discover each other using predictable naming conventions. Each service automatically gets a DNS entry (<service-name>.<namespace>.svc.cluster.local). Without this, applications would need to hardcode IPs, which are dynamic and unreliable. Internal DNS is also used by Kubernetes controllers and admission webhooks. Ensuring that the DNS pods (usually CoreDNS) are running and reachable is critical to the overall stability and availability of applications on OpenShift.

6. How does OpenShift ensure high availability of Ingress Controllers?

Ingress Controllers are typically deployed as pods in a highly available setup, often using a DaemonSet or Deployment with replicas on multiple nodes. An external load balancer (like F5, HAProxy, or cloud-native ELBs) routes external traffic to these Ingress pods. If one Ingress pod fails, traffic is redirected to another healthy pod, ensuring availability. For HA, it's also vital to distribute ingress controllers across availability zones or failure domains. DNS wildcard entries route to the load balancer, making public app exposure seamless.

7. What is the NodePort service type used for, and when should it be avoided?

NodePort exposes a service on a static port (30000–32767) on all nodes, allowing external traffic to reach a service without a load balancer. It's mainly used in development, bare-metal, or lab environments. However, it lacks high availability and dynamic load balancing. If the node goes down, the service becomes unreachable unless failover mechanisms are manually handled. For production environments, it is better to use LoadBalancer or Ingress types for reliable and scalable external access.

8. What is the function of an ExternalName service in OpenShift?

An ExternalName service maps a Kubernetes service to an external DNS name. It allows pods to access external services like databases or APIs using internal DNS, which gets resolved to a real external hostname. This is useful for decoupling application logic from hardcoded external URLs. It doesn't create a proxy or route traffic, just returns the DNS result. From an infra point of view, it simplifies configurations and is often used in hybrid cloud or integration scenarios.

9. How does OpenShift use CoreDNS, and what should be monitored?

CoreDNS is the internal DNS server deployed as dns-default pods. It handles DNS resolution for internal services, pods, and external names. It interacts with the Kubernetes API to provide dynamic DNS records. Infra teams should monitor:

dns-default pod health

DNS response latency

CPU/memory of DNS pods

Log errors like NXDOMAIN or SERVFAIL Issues here can break service discovery, causing app failures. Scaling DNS pods and using node placement strategies improve performance.

10. What happens if DNS wildcard entries are not configured correctly?

If wildcard DNS (e.g., *.apps.cluster.example.com) is misconfigured or missing, application routes created in OpenShift won't be resolvable externally. Users will see DNS resolution errors or 404 pages. It also breaks external access to apps deployed with Routes. Infra teams must ensure that the DNS wildcard entry points to the external IP or FQDN of the Ingress Load Balancer. This setup is critical and often validated during OpenShift installer pre-checks or Day 2 verification steps.

11. How are Routes different from Ingress in OpenShift?

Routes are OpenShift's native way of exposing services to the outside world over HTTP/HTTPS. In contrast, Ingress is a Kubernetes standard. Both use Ingress Controllers behind the scenes (like HAProxy), but Routes offer more OpenShift-specific features such as sticky sessions, edge/re-encrypt/ passthrough TLS termination types. Routes are easier to use in OpenShift environments, especially in disconnected clusters. Infra admins often prefer Routes for simplicity and integration with OpenShift Console, whereas Ingress is used for cross-platform portability.

12. What is a LoadBalancer service and its dependency on cloud platforms?

The LoadBalancer service type provisions an external load balancer via cloud provider APIs (AWS, Azure, GCP). It assigns a public IP and forwards traffic to backend pods. It's simple and reliable but only works in cloud environments. In on-prem environments, external tools like MetalLB or F5 are needed to mimic this behavior. Infra teams must ensure proper cloud integration, IAM roles, and networking (like subnets and routes) to allow automatic provisioning and health checks of LBs.

13. Why is the Ingress Controller deployed in the openshift-ingress namespace?

This namespace is a system project that isolates all Ingress-related components, including HAProxy pods and configuration secrets. It simplifies RBAC, logging, monitoring, and updates related to ingress functionality. Keeping it separate ensures that users don't accidentally modify critical components. Infra teams should protect and monitor this namespace for pod restarts, certificate expiries, and performance metrics. The Ingress Controller also reads the wildcard certificate and routes from all namespaces, making it a cluster-wide resource.

14. What is the purpose of NetworkPolicy in OpenShift v4?

NetworkPolicies define rules to control which pods can communicate with each other. By default, all pods can talk to all other pods. Applying NetworkPolicies allows you to restrict traffic based on labels, namespaces, and ports, enhancing security. For example, you can allow only front-end pods to connect to backend pods. These policies are enforced by the network plugin (like OVN-Kubernetes). Infra admins use them to achieve zero-trust networking within the cluster and meet compliance requirements.

15. How does MTU size impact OpenShift networking performance?

MTU (Maximum Transmission Unit) defines the largest size of a packet that can be transmitted. If OpenShift's pod network MTU is larger than the underlying physical or virtual network, packets will fragment or drop, causing application performance issues. Common MTU sizes are 1450 or 9001 (jumbo frames). Proper alignment avoids overhead and maximizes throughput. It’s important during installation and should be tested with tools like ping -s. Changing MTU post-installation can be risky and disruptive.

16. How do Egress IPs work in OpenShift v4, and why are they useful?

Egress IPs allow specific projects (namespaces) to use a dedicated outbound IP when accessing external resources. This is especially useful for white-listing in firewalls, ensuring predictable source IPs. With OVN-Kubernetes, egress IPs can be dynamically assigned to nodes and used for outbound traffic from selected pods. Infra teams configure them using EgressIP custom resources, and OpenShift automatically handles NAT and failover. It’s critical for secure and auditable internet access, particularly in regulated environments or when integrating with third-party APIs.

17. What are the best practices for DNS failover in OpenShift?

For DNS failover:

Deploy multiple dns-default pods across availability zones

Use liveness and readiness probes to detect failure

Enable metrics monitoring for DNS latency and errors

Configure external DNS with multiple A records or use Route53/Cloud DNS failover policies

On-prem, use DNS servers that support high availability DNS failures can cripple service discovery. Infra teams must ensure DNS is distributed, monitored, and resilient to avoid cascading service outages.

18. How is IPAM (IP Address Management) handled in OpenShift networking?

IPAM in OpenShift is managed automatically by the CNI (Container Network Interface) plugin, which allocates IPs from predefined ranges for pods and services. Each node gets a subnet slice for its pods, and IPs are allocated dynamically. For OVN, OpenShift manages this allocation centrally. Infra teams should size the IP pools to avoid exhaustion, especially in dense clusters or where many short-lived pods are expected. IP exhaustion can prevent pod scheduling and lead to cascading deployment failures.

19. How does OpenShift route external traffic through HAProxy Ingress Controllers?

OpenShift uses HAProxy-based Ingress Controllers to terminate external traffic. When a Route is created, it's configured in HAProxy as a frontend-backend rule. HAProxy listens on ports 80 and 443, terminates TLS (if required), and forwards traffic to service endpoints based on hostname and path rules. Routes can use different TLS termination types like passthrough, edge, or re-encrypt. Infra teams configure the Ingress Controller with appropriate certificates, tuning parameters, and node selectors for performance and security.

20. What are some common reasons why Routes stop working in OpenShift?

Routes may stop working due to:

Ingress Controller pods being down

Wildcard DNS misconfiguration

Certificate issues (expired or misapplied)

Misconfigured Route rules (wrong hostname/path)

Load balancer or firewall blocking port 80/443

Node port conflict Infra teams should check Ingress logs (openshift-ingress), validate DNS records, test connectivity using curl, and inspect HAProxy status pages. Ensuring redundant Ingress pods and using readiness probes can mitigate such outages.

21. How is OpenShift networking different in disconnected (air-gapped) environments?

In disconnected environments, OpenShift cannot access the internet to pull images, resolve public DNS, or use external load balancers. All images must be mirrored locally. DNS resolution relies on internal DNS or enterprise DNS setups. Load balancing is usually done via HAProxy or F5 appliances. Networking must be tightly controlled, with careful routing, firewall rules, and MTU sizing. Infra teams must prepare by creating local mirrors, configuring registry access, and validating all networking components manually before deployment.

22. What tools are available to troubleshoot networking in OpenShift v4?

Common tools include:

oc exec + ping, curl, nslookup inside pods

oc get network, oc get pods -n openshift-network

tcpdump and wireshark for packet tracing

ovn-trace for OVN-based networking

oc adm inspect for cluster-wide dumps

HAProxy router stats pages Infra teams use these to debug issues like dropped packets, failed DNS, route misconfigurations, or pod-to-pod communication failures. OpenShift also integrates metrics via Prometheus and logging via EFK/Logging Operator for deeper analysis.

23. How can you restrict internet access for certain pods or namespaces?

Use EgressNetworkPolicy (in OVN) or egress firewall rules to restrict external access from specific namespaces or pods. Additionally, use service mesh or proxies to enforce fine-grained controls. You can also use firewall/NAT rules at the node or network gateway level. From an infra perspective, define egress IPs only for allowed namespaces and ensure proper logging. This approach helps meet compliance or security rules, especially in financial or healthcare deployments where data exfiltration must be controlled.

24. What is the role of MetalLB in OpenShift bare-metal deployments?

MetalLB is used to provide LoadBalancer-type service support in on-prem or bare-metal OpenShift environments. Since there's no native cloud load balancer, MetalLB assigns external IPs from a pool and forwards traffic to the correct nodes using L2 (ARP) or L3 (BGP) protocols. It works with Ingress or any LoadBalancer service. Infra teams configure IP pools, BGP peers (if needed), and ensure the network allows ARP broadcasting. MetalLB is a lightweight and effective solution for external exposure.

25. How do Routes support TLS termination and what are the types?

OpenShift Routes support three types of TLS termination:

Edge: TLS terminates at the router; traffic to backend is plain HTTP

Passthrough: TLS terminates at the backend pod; router just forwards encrypted traffic

Re-encrypt: TLS is terminated at the router and re-encrypted for the backend Each has its use case based on trust, compliance, and inspection needs. Infra teams manage the wildcard or custom certificates and enforce policies on which termination to allow in each namespace.

26. How does OpenShift handle IPv6 networking?

OpenShift v4 (with OVN-Kubernetes) supports dual-stack networking—IPv4 and IPv6. This allows pods and services to get both IP versions. However, certain features like external DNS, storage, and load balancers must also support IPv6. Infra teams must validate that the underlying OS, CNI, and DNS can handle IPv6 traffic and configure IPv6 addresses and routes. While not mandatory, IPv6 adoption is growing in government and telecom sectors, making dual-stack support a future-ready feature in OpenShift.

27. How is service-to-service communication secured in OpenShift v4?

By default, OpenShift uses a flat network where all services can talk to each other. To secure communication:

Use NetworkPolicies to restrict traffic

Use TLS between services

Employ service meshes (like Istio) for mTLS and access control

Use internal service DNS names for controlled routing Infra teams often combine Kubernetes-native policies with third-party tools (firewalls, proxies) to implement layered security. Monitoring and logging inter-service traffic is also critical for breach detection and incident response.

28. How does OpenShift monitor and alert for networking issues?

OpenShift uses Prometheus and Alertmanager to monitor metrics like DNS latency, router response time, pod-to-pod connectivity, and network plugin health. Preconfigured alerts like “Ingress Controller down” or “DNS pod crashlooping” are enabled by default. Infra teams can add custom rules for packet drops, high router CPU usage, or slow route responses. Integration with external tools (like Grafana, Splunk) provides visibility. Keeping dashboards and alerting updated is essential for quick detection and resolution of networking issues.

29. How do you expose internal OpenShift services only to specific clients or networks?

Use NetworkPolicies, firewall rules, and Route annotations with whitelisting to restrict access. For example, use haproxy.router.openshift.io/ip_whitelist to allow specific IPs on a Route. Services can also be exposed via internal DNS only or placed behind an internal Ingress Controller accessible only on private subnets. Infra teams enforce access control using firewalls, load balancer ACLs, and node-level security groups to ensure services are not unintentionally exposed to the public.

30. What should be part of a pre-installation checklist for OpenShift networking?

A strong pre-install checklist includes:

Validating DNS (forward/reverse) and wildcard entries

Ensuring correct MTU and no overlapping CIDRs

Verifying port openings (e.g., 6443, 443, 80, 2379–2380)

Planning for Load Balancer (cloud or external)

Reserving IP ranges for pods/services

Preparing firewall rules for east-west and north-south traffic

Ensuring time sync across nodes (important for TLS) Infra teams that follow a comprehensive checklist reduce deployment failures and Day-2 surprises significantly.

31. What is OpenShift v4 networking about?

OpenShift v4 networking enables communication between pods, services, and external systems. It includes internal cluster networking, ingress/egress traffic control, DNS resolution, and load balancing for services.

32. Why is networking important in OpenShift v4?

Without a reliable and secure network, pods cannot communicate with each other or with external systems. Networking ensures:

Inter-pod communication

Access to services from users or applications

Security and traffic routing

Performance and scalability

33. What are the key components of OpenShift v4 networking?

SDN (Software Defined Network): OpenShiftSDN, OVN-Kubernetes

Ingress Controller: Routes external traffic to internal services

Services: Abstracts pod endpoints behind a single IP

DNS: Resolves internal service names

Load Balancer: Distributes external traffic

NetworkPolicy: Controls pod-to-pod traffic

34. What are the two main network plugins available in OpenShift v4?

OpenShiftSDN: Traditional plugin, deprecated in newer releases

OVN-Kubernetes: Default plugin in newer OpenShift versions, supports better scalability and security

35. What is the Pod Network in OpenShift?

Every pod in OpenShift gets a unique IP from a predefined CIDR range. All pods can communicate with each other by default unless restricted by network policies.

36. What is a Service in OpenShift networking?

A service is an abstraction layer that defines a logical set of pods and a policy to access them. Services are assigned a stable virtual IP (ClusterIP) for communication.

37. What types of services exist in OpenShift?

ClusterIP: Default; accessible only within the cluster

NodePort: Exposes service on a static port on each node

LoadBalancer: Exposes service externally using cloud/native load balancer

ExternalName: Maps service to external DNS name

38. How does DNS work in OpenShift v4?

OpenShift provides internal DNS that resolves service names to ClusterIP. It uses dns-default pods running CoreDNS.

39. Why do we need internal DNS in OpenShift?

Resolves service names (e.g., myapp.myproject.svc.cluster.local)

Facilitates service discovery

Reduces hardcoding of IPs

40. What are some important DNS considerations?

Ensure the base domain is correctly set (e.g., apps.clustername.example.com)

DNS wildcard entry (*.apps) should point to Ingress Load Balancer

Ensure external DNS is properly integrated for public access

41. What is a Load Balancer in OpenShift?

A Load Balancer distributes incoming traffic across multiple endpoints (pods or nodes). It is often used with NodePort or Ingress Controllers.

42. What is an Ingress Controller in OpenShift v4?

Ingress Controller is a component that manages incoming HTTP/HTTPS traffic and routes it to internal services using defined rules.

43. What kind of load balancers can be used in OpenShift v4?

Cloud-native LB (AWS ELB, Azure LB, GCP LB)

HAProxy-based external LB

MetalLB (for bare-metal)

F5 Big-IP, NGINX LB

44. Which ports must be opened for OpenShift v4 to function?

Here are the critical ports:

API Server: 6443 (HTTPS)

etcd: 2379-2380

Ingress traffic: 80 (HTTP), 443 (HTTPS)

SSH (for access): 22

Node communication: 10250 (kubelet), 30000-32767 (NodePort range)

45. Why are port requirements important in OpenShift?

Without proper port configuration:

Nodes won't register

Ingress won't work

Apps may become inaccessible

Monitoring and metrics collection can fail

46. What are key networking considerations for Infra teams?

Choose correct network plugin (OVN preferred)

Assign appropriate CIDRs:

Cluster Network (pods)

Service Network (ClusterIPs)

Enable MTU and jumbo frame support if needed

Monitor and manage bandwidth

Configure firewall and security zones correctly

Ensure DNS and Load Balancer are in place before installation

47. What is the default MTU size for OpenShift networks?

Default is typically 1450 or 9001 (depending on cloud/infra), but it should align with your physical or virtual network MTU settings.

48. What is a NetworkPolicy?

NetworkPolicy is a Kubernetes resource that allows controlling traffic flow at the IP address or port level between pods.

49. Do OpenShift clusters need a wildcard DNS entry?

Yes. A wildcard DNS (*.apps.<cluster-domain>) is required to route all application routes to the Ingress Controller.

50. Can multiple ingress controllers be configured in OpenShift?

Yes, OpenShift supports custom Ingress Controllers (e.g., public and private routes using different routers or certificates).